1 Introduction

As FPGA development shifts toward remote hardware execution and testing, managing the physical infrastructure becomes a bottleneck. This is especially true in a mixed-architecture environment. At the lab, our hardware farm consists of standard Intel Cyclone V (DE-SoC) boards alongside a cutting-edge Intel Agilex architecture board.

Traditionally, assigning tasks in a farm like this relies on static routing. A student has to manually specify the specific board they want to target. If that board is powered off, or offline, the deployment fails. Hardcoding routing logic becomes less viable as new hardware is added. To solve this, we developed an intelligent, fully automated control plane that separates the physical hardware from the user, utilizing a Large Language Model (LLM) to dynamically route compilation tasks.

2 The Control Plane

The foundation of this infrastructure is a Python microservice built with Flask. It operates as a background task on the primary compilation server, where the API listens for HTTP requests over the network.

To ensure a dynamic program in a multi-user environment, the system utilizes Python's threading.Condition() lock. This function tracks the status (idle, busy, or quarantined), power state, and a continuous "heartbeat" timestamp for every physical FPGA. By keeping state modifications within thread locks, the system prevents conditions where two simultaneous deployments might attempt to use the same physical hardware.

3 API Design and Specifications

The system keeps the routing logic separate from the user interface by using a setup similar to a REST API. It utilizes five main endpoints, with each one taking care of a different step in handling a hardware task.

- **3.1 Task Initialization: POST /route_task** When a new hardware task is submitted, a JSON payload containing the task identifier and project metadata is sent. Upon receiving this payload, the API passes the data to the local LLM engine. If the LLM successfully routes the task, the API locks the thread, increments the pending queue for the target architecture, and allocates an available physical instance. If the board is currently powered down, the API triggers a hardware execution command to power on the smart Wi-Fi plug associated with that board.
- **3.2 Resource Deallocation: POST /release_board** Once a board finishes programming and the active user is done, it must be released for the next user. The program posts to this endpoint with the physical instance name. This endpoint implements smart power management. Before powering down the released board, the API checks the pending queue for that specific architecture. If the queue is empty, the board is powered down to conserve energy. If tasks are waiting, the power remains on, and the thread lock calls notify_all() to wake all waiting tasks in the queue.

- **3.3 Board Monitoring: GET /status and POST /heartbeat** To provide real-time visualization without freezing the client GUI, a GET /status endpoint returns the entire dashboard state dictionary, allowing users to render live execution times and power states. To prevent frozen tasks from holding up the hardware, the system utilizes a /heartbeat endpoint. The client must ping this endpoint periodically through programming. A background watchdog thread continuously polls the last heartbeat timestamps. If a board exceeds the timeout threshold without a change to the board, the watchdog forcefully releases the board, updates its status to idle, and powers it down. This ensures that if a student forgets about their board, the farm still remains highly available.
- **3.4 Emergency Quarantine: POST /emergency_stop** An authorized endpoint allows administrators to manually isolate specific boards in the farm. This forces the target hardware into a quarantined status and unpowers it.

4 Task Routing via Local LLM

The most important part in this architecture is the replacement of static routing logic with a local LLM. To determine which board should receive which task, Ollama is integrated directly onto the main server. This allows the execution of the Qwen 2.5 (7b) model locally.

When /route_task is called, the API packages the incoming JSON metadata into a structured system prompt. The AI acts as an automated routing agent, evaluating constraints (e.g., routing high resource utilization to an Agilex 10, while sending basic testing to an idle DE-SoC). Crucially, the LLM is forced via prompt engineering to return its decision as a JSON object. By keeping the inference engine strictly on our local server, the execution of routing tasks to boards becomes significantly faster. To ensure reliability, this automated routing also has error handling. If the LLM produces an unparseable or failed response, the system returns a bad request error rather than attempting to manually compile.

5 Conclusion

By combining a Flask API, safe queue management, and a local AI routing agent, the FPGA Farm becomes a collection of boards that are efficient and automated. Because the AI agent and the hardware execution are based on an API system, the control plane can easily be migrated to a standalone edge device, such as a Raspberry Pi 5, serving as a dedicated node for the lab.